

Technical Requirements Document

AI Room Reshuffle & Redesign App

Working Name: ReSpace AI

Mobile-only Flutter application for iOS and Android

Storage Provider: Cloudflare R2

This TRD defines the mobile app architecture, backend services, AI/video processing pipeline, database models, APIs, background jobs, security, scalability, testing, and MVP implementation plan.

1. Document Overview

Purpose

This Technical Requirements Document defines the technical architecture, system components, data models, APIs, AI pipeline, storage design, background processing strategy, security requirements, scalability plan, and implementation roadmap for the AI Room Reshuffle & Redesign application.

Technical Summary

ReSpace AI is a mobile-first video-to-room-planning platform. Users record or upload a room video from a Flutter mobile app. The backend extracts frames, detects visible items and room structure, allows user correction, generates reshuffle layouts, and stores project outputs for future editing.

Technical Prioritization

The highest priority is not photorealistic generation. The highest priority is a trustworthy pipeline: video upload -> frame extraction -> item detection -> user correction -> layout generation -> saved final plan. Measurement estimation, product recommendations, AR, and 3D editing should come after MVP validation.

2. Technical Goals and Non-Goals

Technical Goals

Reliable video upload, AI-assisted room understanding, user correction layer, multiple layout generation, persistent project storage, asynchronous processing, and scalable architecture.

MVP Non-Goals

Real-time AR rendering, native 3D room reconstruction, CAD-level measurement precision, automated checkout, multi-room floor plans, designer marketplace, collaborative editing, complex drag-and-drop 3D editor, and full LiDAR-based measurement are out of MVP scope.

Build Principle

Start with a focused reshuffle workflow. Do not build redesign, shopping, exact measurements, AR, and marketplace features at the same time. That would create technical bloat before core value is proven.

3. Recommended Technology Stack

3.1 Required Mobile Frontend: Flutter

This product is mobile-only. The core user experience must be built as a Flutter app for iOS and Android. There is no web application, web dashboard, browser upload flow, Next.js product experience, or React Native implementation in scope for the app.

Area	Required Choice	Reason
Mobile app	Flutter + Dart	Single codebase for iOS and Android, strong UI performance, good camera/video support.
State management	Riverpod, Bloc, or Provider	Keeps upload, processing, project state, and UI steps organized.
Navigation	GoRouter or AutoRoute	Supports tab navigation plus multi-step project flows.
API client	Dio or HTTP package	Handles authenticated requests, retries, upload completion calls, and polling.
Media capture	camera, image_picker, video_player	Supports in-app recording, gallery upload, and preview.
Secure storage	flutter_secure_storage	Stores access/refresh tokens securely.
Light preferences	shared_preferences	Stores non-sensitive preferences such as theme and unit system.
Image caching	cached_network_image	Improves generated image loading and project browsing.

3.2 Backend

Recommended backend: Python FastAPI. Python is better suited for AI/video processing, computer vision, FFmpeg/OpenCV workflows, background workers, and AI provider orchestration. The backend should be a modular monolith for MVP, not premature microservices.

Component	Recommended Technology
API framework	FastAPI + Pydantic
Database ORM	SQLAlchemy or SQLModel
Migrations	Alembic
Queue workers	Celery or Dramatiq
Queue broker/cache	Redis
Database	PostgreSQL
Media storage	Cloudflare R2

3.3 Database

Use PostgreSQL as the system of record. It supports relational project data, JSONB metadata for AI outputs, future pgvector embeddings, and optional PostGIS if spatial room features become advanced.

3.4 Required Object Storage: Cloudflare R2

Cloudflare R2 is the required cloud object storage provider for uploaded room videos, extracted frames, thumbnails, generated layout images, generated redesign images, top-down layouts, and final plan exports.

R2 Requirement	Details
Access model	Private buckets by default. Use signed upload and signed read URLs.
Upload flow	Mobile app uploads directly to R2 using a signed upload URL generated by backend.
Read flow	Backend validates ownership before generating expiring signed read URLs.
Server access	Backend and workers use server-side R2 credentials only.
URL policy	Do not expose permanent public URLs for private room videos.
Cleanup	Deleting a project should delete or enqueue cleanup for related R2 objects.

Recommended R2 prefix structure:

```
respace-media/  
  users/{user_id}/projects/{project_id}/videos/  
  users/{user_id}/projects/{project_id}/frames/  
  users/{user_id}/projects/{project_id}/generated/  
  users/{user_id}/projects/{project_id}/exports/
```

4. High-Level System Architecture

The system should use a modular monolith plus background workers for MVP. Split services only when scale and team size justify it.

4.1 Core Components

- Flutter mobile app
- Backend API service
- Authentication service/integration
- Project service
- Media service
- AI processing orchestrator
- Background worker service
- Cloudflare R2 storage
- PostgreSQL database
- Redis queue/cache
- Realtime or polling status service
- Product recommendation service later
- Payment/billing service later

4.2 Architecture Flow

Flutter Mobile App

- > Backend API Service
 - > Auth / Project / Media / Preferences / Generation APIs
 - > PostgreSQL
 - > Cloudflare R2 signed URLs
 - > Redis Queue
 - > Background Workers
 - > Video Processing
 - > Frame Extraction
 - > Object Detection
 - > Room Analysis
 - > Layout Generation
 - > Image Generation
 - > Product Matching later
 - > AI Providers / ML Models

4.3 MVP Deployment Architecture

- Mobile app: Flutter app distributed to iOS and Android test users
- Backend API: FastAPI on Render, Fly.io, Railway, AWS ECS, or Google Cloud Run
- Database: managed PostgreSQL
- Queue/cache: managed Redis
- Storage: Cloudflare R2
- Workers: same backend codebase deployed as a worker process

5. System Modules

Authentication Module

Handles signup, login, logout, password reset, OAuth optional, secure token validation, user deletion, and ownership checks. Use Supabase Auth, Firebase Auth, Clerk, Auth0, or custom JWT only if necessary.

Project Module

Creates, updates, deletes, and lists projects. Stores project mode, room type, processing status, thumbnail, and ownership.

Media Module

Generates R2 signed upload URLs, validates uploads, stores media records, extracts frames, creates thumbnails, and deletes media on request.

Frame Extraction Module

Uses FFmpeg and OpenCV to extract frames, remove blurry or duplicate frames, score quality, and keep 8-15 representative frames for AI analysis.

Object Detection Module

Detects furniture and major room objects, returns labels/confidence/bounding boxes, and prepares item data for user correction.

Room Structure Module

Detects walls, floor, doors, windows, open zones, and fixed structural features. MVP can keep this rough; redesign mode requires more detail.

Measurement Module

Estimates measurements only for redesign mode. Measurements must be labeled as estimated unless user-verified or calibrated.

Layout Generation Module

Generates multiple reshuffle layouts using confirmed items, fixed/movable constraints, goals, and room context.

Image Generation Module

Generates visual room concepts and top-down layout images. Images are suggestions, not exact engineering plans.

Product Recommendation Module

Post-MVP module for furniture/decor search, dimensions, fit checks, budgets, stores, and shopping list generation.

Status Module

Tracks uploading, processing, detection, review, generation, completion, and failure states. Polling is enough for MVP; WebSockets/SSE can come later.

6. Data Model

The database should store structured project and AI output data. Do not store only raw text responses from AI models. Store structured JSON for layouts, item changes, measurement sources, fit status, and final plans.

User

- > Project
 - > MediaAsset
 - > ExtractedFrame
 - > DetectedItem
 - > Measurement
 - > PreferenceSet
 - > GeneratedDesign
 - > DesignItemChange
 - > ProductRecommendation
 - > FinalPlan

Entity	Purpose	Important Fields
users	Stores account identity and auth linkage.	id, email, name, auth_provider, created_at, deleted_at
projects	Stores room project metadata and state.	id, user_id, name, room_type, mode, status, thumbnail_url
media_assets	Stores uploaded/generated media references.	project_id, user_id, type, storage_key, mime_type, file_size, status
extracted_frames	Stores selected video frames.	media_asset_id, storage_key, timestamp, blur_score, quality_score
detected_items	Stores AI/user item inventory.	label, user_label, confidence, source, movable, bounding_boxes, metadata
measurements	Stores estimated/corrected dimensions.	measurement_type, value, unit, source, confidence
preference_sets	Stores user goals and constraints.	goal, style, budget, colors, stores, fixed_item_ids
generated_designs	Stores generated layouts/concepts.	type, title, description, image_asset_id, difficulty, fit_confidence
design_item_changes	Stores move instructions per layout.	item_name, change_type, before_location, after_location, instruction
product_recommendations	Stores future redesign products.	product_name, store, price, dimensions, fit_status, product_url
final_plans	Stores selected final output.	generated_design_id, summary, checklist, warnings, export_asset_id

7. API Requirements

Use REST for MVP. It is simple, debuggable, mobile-friendly, and sufficient for project/media/generation workflows. GraphQL is unnecessary for the first version.

Area	Endpoint Examples	Purpose
Auth	POST /auth/signup, POST /auth/login, GET /auth/me	Create account, login, validate current user.

Projects	POST /projects, GET /projects, GET/PATCH/DELETE /projects/{id}	Create, view, update, and delete room projects.
Media	POST /projects/{id}/media/upload-url, POST /media/{id}/complete	Create R2 signed upload URL and trigger processing.
Processing	GET /projects/{id}/processing-status, POST /retry-processing	Return processing progress and retry failed jobs.
Items	GET/POST/PATCH/DELETE /projects/{id}/items	Review and correct detected items.
Measurements	GET/PATCH /measurements, POST /measurements/calibrate	Review, correct, and calibrate measurements.
Preferences	GET/PUT /projects/{id}/preferences	Save reshuffle/redesign preferences.
Generation	POST /generate-layouts, GET /designs, POST /designs/{id}/customize	Generate, retrieve, select, and customize layouts.
Final Plan	POST /final-plan, GET /final-plan, POST /final-plan/export	Create and retrieve the final actionable plan.

8. Background Job Pipelines

8.1 Video Processing Pipeline

Upload Complete -> Validate Video -> Extract Frames -> Score/Select Frames -> Detect Objects -> Analyze Room Structure -> Save Detection Results -> Project Status: needs_review

8.2 Reshuffle Generation Pipeline

User Confirms Items + Preferences -> Generate Layout Reasoning -> Generate Layout Images -> Generate Top-Down Layouts -> Generate Move Instructions -> Save Generated Designs -> Project Status: completed

8.3 Redesign Generation Pipeline

User Confirms Measurements + Preferences -> Generate Design Concepts -> Search Products -> Check Product Fit -> Generate Room Renders -> Generate Shopping List -> Save Redesign Concepts

8.4 Job Retry Strategy

- Video validation: 1 retry
- Frame extraction: 2 retries
- AI detection: 2 retries
- Image generation: 2 retries
- Product search: 1 retry
- Failed jobs must store reason, attempt count, and last attempted timestamp.

9. AI Pipeline Requirements

AI Input Preparation

Before model calls, prepare structured context: room type, mode, selected frames, corrected items, fixed/movable constraints, user goals, style preferences, and measurement data if available.

Object Detection Strategy

Use a hybrid approach: vision-capable model for semantic room understanding plus object detection/vision API for item labels and bounding boxes. User correction remains the final truth.

Layout Reasoning Strategy

The layout model should produce validated structured JSON, not loose paragraphs. Store title, goal alignment, difficulty, items moved, items unchanged, pros, cons, and setup steps.

Image Generation Strategy

Generate visuals from confirmed structured layout plans. Do not let the image model invent impossible furniture placements or ignore fixed items.

Measurement Estimation Strategy

Treat measurement estimation as assistive, not authoritative. User-corrected measurements override AI estimates. Use confidence levels: low, medium, high, user_verified.

10. Flutter Mobile App Technical Requirements

10.1 Navigation Structure

Use bottom tabs for the main app and stack navigation for project creation flows.

Bottom Tabs:

- Home
- Projects
- Explore
- Profile

Project Flow:

Home -> Mode Selection -> Capture Instructions -> Upload/Record -> Processing -> Detected Items
Review -> Preferences -> Results -> Design Detail -> Final Plan

Redesign inserts:

Measurement Review -> Calibration -> Product Recommendations

10.2 Flutter State Management

- Local UI state: current step, temporary form data, modal state, upload progress, camera state.
- Server state: projects, processing status, detected items, measurements, generated designs, final plans.
- Secure local state: access tokens, refresh tokens, auth session data in flutter_secure_storage.
- Non-sensitive local preferences: theme, measurement unit, capture-instruction seen flag in shared_preferences.

10.3 Upload UX Requirements

- Show file validation errors.
- Show upload progress.
- Allow retry on failed upload.
- Show processing progress and clear stage messages.
- Allow user to leave and return while backend processing continues.

11. Backend Technical Requirements

- All project/media/generation APIs require authentication unless guest mode is explicitly enabled.
- Every project-scoped endpoint must enforce user ownership.
- Validate file type, file size, project mode, room type, units, budget values, item names, and generation count.
- Rate limit expensive endpoints such as upload URL creation, video retry, layout generation, customization, and product search.
- Use idempotency for upload complete, start processing, layout generation, and final design selection.
- Use consistent error responses with code, message, and details.

12. Security and Privacy Requirements

- Room videos are sensitive private media. Use private R2 buckets and signed URLs only.
- Never expose permanent public URLs for private videos.
- Do not use user videos for model training without explicit opt-in.
- Allow users to delete uploaded videos, extracted frames, projects, generated designs, and account data.
- Do not log raw media, signed URLs, secrets, or sensitive visual details.
- Mitigate prompt injection by using system prompts, structured output schemas, and AI output validation.

13. Performance Requirements

- Mobile uploads should go directly to R2 using signed URLs. The backend should not proxy large files.
- Target frame extraction under 60 seconds for normal videos.
- Target object detection under 90 seconds.

- Target layout generation under 2-4 minutes.
- The Flutter app must remain responsive during upload and processing.
- Cache generated images and project metadata safely.

14. Observability Requirements

- Log API request IDs, job IDs, processing stage transitions, failure reasons, AI provider latency, and storage errors.
- Track upload success rate, video processing failure rate, average processing time, detection completion rate, layout generation success rate, AI cost per project, user correction rate, and saved design rate.
- Use Sentry or equivalent for frontend, backend, and worker errors.

15. Cost Control Requirements

- Limit video duration and file size.
- Limit number of frames sent to AI.
- Cache intermediate outputs.
- Limit free generations.
- Reuse detection output where possible.
- Use credits or paid tier for expensive repeated generations.

16. Scalability Plan

- MVP: one API service, one worker service, managed PostgreSQL, managed Redis, Cloudflare R2.
- Growth: split workers by job type: media, detection, generation, product.
- High scale: dedicated queues, autoscaling workers, CDN for generated images, database read replicas, provider cost routing.

17. Testing Requirements

Test Type	Coverage
Unit tests	API validation, authorization, measurement conversion, fit calculation, prompt construction, output schema validation.
Integration tests	Project creation, R2 upload lifecycle, processing status, item editing, preference saving, job creation, final plan creation.
Worker tests	Frame extraction, job retries, failed job handling, AI output parsing, R2 read/write.
Flutter tests	Project flow navigation, upload UI, item review editing, preference selection, results display, error states.
Manual QA videos	Bright room, dark room, shaky video, small bedroom, living room, cluttered room, empty room, mirror, people/pets, missing wall.

18. DevOps and CI/CD

Repository Structure

```

respace-ai/
  apps/
    mobile_flutter/
  services/
    api/
    worker/
  packages/
    shared-types/
    ai-prompts/
    storage-client/
  infra/
    docker/

```


terraform/
docs/
PRD.md
TRD.md

- Environments: local, staging, production.
- Local uses Docker Compose for PostgreSQL and Redis.
- Staging uses real R2, real database, limited AI keys, and test users.
- Production uses managed infrastructure, monitoring, backups, rate limits, and billing controls.
- CI should run linting, type checks, unit tests, API tests, migration checks, and security scanning.
- CD should support staging deployment, production release approval, and rollback.

19. Feature Flags

- Redesign mode
- Measurement estimation
- Product recommendations
- Guest mode
- High-resolution generations
- AR preview
- Payment system

20. MVP Implementation Plan

Phase	Build Items
Phase 0: Foundation	Repo setup, auth, database schema, project CRUD, R2 integration, signed URLs, worker setup, Flutter navigation.
Phase 1: Media Pipeline	Video recording/upload, upload complete endpoint, processing status, frame extraction, thumbnail generation, failure handling.
Phase 2: Object Detection and Review	AI object detection, detected items table, item review screen, add/edit/delete item, movable/fixed toggle.
Phase 3: Reshuffle Generation	Preference screen, layout reasoning prompt, structured generation, layout images, top-down layouts, results screen.
Phase 4: Final Plan	Select design, generate move checklist, final plan screen, save project, project history.
Phase 5: Beta Hardening	Rate limits, error monitoring, retries, better failure states, cost controls, user feedback/rating.

21. Future Implementation Plan

- Version 2: measurement estimation, review, correction, calibration, and unit conversion.
- Version 3: redesign mode with style, budget, and existing item keep/remove flow.
- Version 4: product recommendations, dimension parsing, fit matching, shopping list, store filters.
- Version 5: 2D drag-and-drop layout editor, room zones, item locking, manual placement.
- Version 6: AR preview, 3D reconstruction, LiDAR support where available, live furniture overlay.

22. Critical Technical Risks

Risk	Severity	Mitigation
Video-based measurements are inaccurate	High	Label as estimated, require user correction, add calibration, avoid fit guarantees.
Object detection is inconsistent	High	Use user correction, limit MVP categories, let users add missing items.
AI image output ignores constraints	High	Use structured layout reasoning first, generate images from approved plan, validate metadata.
Processing cost becomes too high	High	Limit frame count, limit video duration, cap free generations, cache outputs.
Long processing creates bad UX	Medium	Use async jobs, progress states, notifications later, allow leave-and-

		return.
Product data is unreliable	Medium-High	Delay product recommendations, require dimensions, show fit confidence, use trusted data sources.

23. Technical Decisions

- Use Flutter for the only core app experience.
- Use Cloudflare R2 as required object storage.
- Use modular monolith plus workers for MVP.
- Use async processing for all video and AI-heavy jobs.
- Use signed R2 URLs for upload and read access.
- Make AI outputs editable by design.
- Build reshuffle before redesign.
- Store structured AI outputs, not only natural language responses.

24. MVP Acceptance Criteria

1. User can create an account.
2. User can create a reshuffle project.
3. User can record or upload a room video from the Flutter app.
4. Video is stored securely in Cloudflare R2.
5. Frames are extracted asynchronously.
6. AI detects basic furniture/items.
7. User can edit detected items.
8. User can mark items fixed or movable.
9. User can select reshuffle preferences.
10. User can trigger layout generation.
11. System generates at least 3 layout options.
12. User can view generated layouts.
13. User can select a layout.
14. System creates a final move checklist.
15. User can save and reopen the project.
16. Failed jobs show clear user-facing errors.
17. User cannot access another user's projects.
18. Media files are private and accessed through signed URLs.

25. Final Technical Recommendation

The app should be built as a mobile-only AI room planning platform using Flutter for iOS and Android, a FastAPI backend, PostgreSQL database, Cloudflare R2 for media storage, Redis-backed background jobs, and modular AI provider integrations.

19. Build the Flutter mobile app foundation.
20. Build video recording and upload to Cloudflare R2 using signed URLs.
21. Build backend video processing with frame extraction.
22. Build item detection with user correction.
23. Build reshuffle generation.
24. Build saved projects and final plans.
25. Add measurement estimation only after the core flow works.
26. Add redesign and product recommendations only after measurement correction is reliable.

Blunt engineering call: the biggest mistake would be trying to build the full “AI interior designer with shopping and exact measurements” immediately. The product should first win by doing one thing well: turn a real room video from a mobile app into practical reshuffle plans users can understand and act on.